Redis: structuri de date si cheat sheet

Oct 22, 2026 · @Radu

Cum gandesti in Redis

Redis e un dictionar urias tinut in RAM: fiecare cheie are un singur tip de valoare, iar fiecare tip are comenzile lui. Nu exista tabele, `WHERE` sau `JOIN`. Filtrarea o faci prin felul in care alegi numele cheilor.

- **Cheia e query-ul.** In loc de `SELECT ... WHERE method='POST' AND path='/login'` construiesti cheia `rl:rules:POST:/login` si o citesti direct, in O(1).
- **Namespace cu `:`.** Conventia e `aplicatie:tip:id`, de exemplu `rl:rule:9001` si `rl:cnt:...`. Redis nu stie de foldere; `:` e doar conventie, dar te ajuta la `SCAN` si la ACL-uri.
- **TTL e per cheie, nu per element.** `EXPIRE` se pune pe toata cheia. Un camp dintr-un hash sau un element dintr-un set nu expira separat (exceptie: `HEXPIRE` pe campuri de hash, din Redis 7.4).
- **O comanda e atomica.** Redis executa comenzile una cate una, pe un singur thread. Pentru mai multe comenzi atomice folosesti `MULTI/EXEC` sau un script Lua.
- **Cheia apare cand scrii si dispare cand e goala.** `INCR` pe o cheie inexistenta porneste de la 0. Un hash, list, set sau zset din care stergi ultimul element dispare singur.
- **Tot ce iese din Redis e string.** Numerele vin ca text: `GET` iti da `"5"`, nu `5`. In Lua le treci prin `tonumber`.

Structurile de date

Pentru rate limiting iti ajung cinci tipuri: String pentru contoare, Hash pentru reguli, Sorted Set pentru ferestre glisante, plus Set si List pentru cazuri mai rare.

Tip	Ce e, in termeni C#	Comenzi de baza	La ce il folosesti la rate limiting
String	`string`, `byte[]` sau numar (`long` / `double`)	`SET` `GET` `INCR` `INCRBY` `DECR` `SETNX`	Contorul de fixed window: `rl:cnt:...` cu `INCR` + `EXPIRE`

Tip	Ce e, in termeni C#	Comenzi de baza	La ce il folosesti la rate limiting
Hash	`Dictionary<string,string>` sub o cheie	`HSET` `HGET` `HGETALL` `HDEL` `HINCRBY`	O regula (`rl:rule:9001`); starea unui token bucket (`tokens` , `last`)
List	`LinkedList<string>` : push/pop la capete	`LPUSH` `RPUSH` `LPOP` `LRANGE` `LTRIM` `LLEN`	Ultimele N evenimente (log de incercari esuate), cozi simple
Set	`HashSet<string>` : unic, fara ordine	`SADD` `SREM` `SISMEMBER` `SMEMBERS` `SCARD`	Blocklist/allowlist de IP-uri; IP-uri distincte pe un cont
Sorted Set (ZSET)	`SortedSet` de (membru, score), unic pe membru, sortat dupa score	`ZADD` `ZRANGE` `ZRANGEBYSCORE` `ZREMRANGEBYSCORE` `ZCARD` `ZSCORE`	Indexul de reguli ordonat dupa `Order` ; sliding window log, cu score = timestamp

Mai sunt cateva tipuri pe care le vei intalni mai rar:

- **Bitmap** e un String folosit ca vector de biti (`SETBIT` , `BITCOUNT`). E bun pentru flaguri compacte, de exemplu "userul X a fost activ in ziua Y".
- **HyperLogLog** numara aproximativ elemente distincte in 12 KB fixi (`PFADD` , `PFCOUNT`). Exemplu: cate IP-uri distincte au lovit `/login` azi.
- **Stream** e un log append-only cu consumer groups (`XADD` , `XREADGROUP`). E bun pentru evenimente de securitate procesate asincron, de exemplu de un detector de anomalii.

Exemplu concret pentru Sorted Set, ca e cel mai putin intuitiv: membrul e unic, iar score-ul il sorteaza.

```
ZADD rl:log:1.2.3.4 1729580000123 "req-a"     # score = timestamp in ms
ZADD rl:log:1.2.3.4 1729580000456 "req-b"
ZREMRANGEBYSCORE rl:log:1.2.3.4 0 1729579940456   # sterge ce e mai vechi de
60 s
ZCARD rl:log:1.2.3.4                              # cate requesturi au ramas
in fereastra
```

Ce structura pentru ce algoritm

Fixed window merge cu un String si `MULTI/EXEC`. Restul algoritmilor citesc o valoare si decid pe baza ei, deci au nevoie de Lua ca sa ramana atomici.

Algoritm	Structura	Pasii pe request	Memorie per client	Problema
Fixed window	String	`INCR` + `EXPIRE NX`, compari cu limita	1 numar	Burst la granita: 2x limita in ~2 s, la finalul unei ferestre si inceputul urmatoarei
Sliding window log	Sorted Set (score = timestamp ms)	`ZREMRANGEBYSCORE` vechi, `ZCARD`, `ZADD` daca e sub limita, `EXPIRE`	1 element per request	Exact, dar memoria creste cu limita (1000 req = 1000 elemente)
Sliding window counter	2 Stringuri (fereastra curenta + precedenta)	`GET` ambele, estimezi `prev * (1 - progres) + curr`, `INCR`	2 numere	Aproximativ, dar ieftin; e varianta folosita des in productie
Token bucket	Hash (`tokens`, `last_refill`)	Calculezi tokenii adaugati de la `last`, scazi 1 daca ai, `HSET`	2 campuri	Permite burst controlat; cere ora curenta (`redis.call('TIME')` in Lua)

Pentru Authnomaly: la `/login` ia fixed window acum si sliding window counter cand vrei sa scapi de burst-ul de la granita. Token bucket se potriveste mai bine la API-uri, unde vrei sa permiti burst-uri scurte.

Cheat sheet de comenzi

Intri in consola cu `docker exec -it my-redis redis-cli`. Coloana C# e metoda echivalenta din StackExchange.Redis pe `IDatabase` (toate au si varianta `Async`).

Comanda	Ce face	StackExchange.Redis
Chei		
`EXISTS k`	1 daca exista cheia	`KeyExists`
`DEL k1 k2`	Sterge chei	`KeyDelete`

Comanda	Ce face	StackExchange.Redis
`TYPE k`	Tipul valorii (string, hash, zset…)	`KeyType`
`EXPIRE k 60 [NX]`	TTL in secunde; `NX` = doar daca nu are deja	`KeyExpire(k, ttl, ExpireWhen.HasNoExpiry)`
`TTL k`	Secunde ramase; -1 = fara TTL, -2 = nu exista	`KeyTimeToLive`
`PERSIST k`	Scoate TTL-ul	`KeyPersist`
`SCAN 0 MATCH rl:* COUNT 100`	Itereaza cheile fara sa blocheze serverul	`server.Keys(pattern: "rl:*")`
`RENAME k k2`	Redenumeste	`KeyRename`
String		
`SET k v EX 60 NX`	Seteaza cu TTL, doar daca nu exista (lock simplu)	`StringSet(k, v, ttl, When.NotExists)`
`GET k`	Citeste; nil daca lipseste	`StringGet`
`INCR k` / `INCRBY k 5`	+1 / +5 atomic, porneste de la 0	`StringIncrement`
`MGET k1 k2`	Mai multe chei intr-un drum	`StringGet(RedisKey[])`
Hash		
`HSET k f1 v1 f2 v2`	Seteaza campuri	`HashSet(k, HashEntry[])`
`HGET k f`	Un camp	`HashGet`
`HGETALL k`	Toate campurile	`HashGetAll`
`HDEL k f`	Sterge un camp	`HashDelete`
`HINCRBY k f 1`	+1 atomic pe un camp	`HashIncrement`
List		
`LPUSH k v` / `RPUSH k v`	Adauga la inceput / sfarsit	`ListLeftPush` / `ListRightPush`
`LRANGE k 0 -1`	Toate elementele	`ListRange`

Comanda	Ce face	StackExchange.Redis
`LTRIM k 0 99`	Pastreaza doar primele 100	`ListTrim`
Set		
`SADD k m` / `SREM k m`	Adauga / scoate	`SetAdd` / `SetRemove`
`SISMEMBER k m`	E membru?	`SetContains`
`SMEMBERS k` / `SCARD k`	Toti membrii / cati sunt	`SetMembers` / `SetLength`
Sorted Set		
`ZADD k score m`	Adauga sau actualizeaza score-ul	`SortedSetAdd`
`ZRANGE k 0 -1 WITHSCORES`	Dupa rang, crescator	`SortedSetRangeByRankWithScores`
`ZRANGEBYSCORE k min max`	Dupa interval de score	`SortedSetRangeByScore`
`ZREMRANGEBYSCORE k min max`	Sterge un interval de score	`SortedSetRemoveRangeByScore`
`ZCARD k` / `ZSCORE k m`	Cati membri / score-ul unui membru	`SortedSetLength` / `SortedSetScore`
Atomicitate		
`MULTI` … `EXEC`	Tranzactie: comenzile ruleaza grupat	`CreateTransaction()` + `ExecuteAsync()`
`EVAL "script" 1 key arg`	Ruleaza Lua atomic	`ScriptEvaluate`
Diagnoza		
`MONITOR`	Afiseaza live tot ce primeste serverul (doar in dev)	n/a
`INFO memory`	Memorie folosita	`server.Info("memory")`
`CONFIG GET maxmemory-policy`	Politica de evictie	`server.ConfigGet`
`SLOWLOG GET 10`	Comenzile lente	`server.SlowlogGet`

Comanda	Ce face	StackExchange.Redis
`MEMORY USAGE k`	Cati bytes ocupa o cheie	`server.Execute("MEMORY", "USAGE", k)`
`FLUSHDB`	Sterge tot din DB-ul curent (doar in dev!)	`server.FlushDatabase`

`MONITOR` e cel mai util cand inveti: tii o consola cu el deschisa, dai requesturi in aplicatie si vezi exact ce comenzi trimite codul tau.

Capcane si plan de invatat

Cele mai multe buguri cu Redis vin din cinci greseli. Toate au aparut deja in codul de rate limiting.

- **`KEYS *` in productie.** Blocheaza tot serverul cat parcurge cheile. Foloseste `SCAN`.
- **Read-modify-write din C#.** `GET`, apoi +1 in C#, apoi `SET` pierde incrementari cand vin requesturi in paralel. Foloseste `INCR`, `MULTI/EXEC` sau Lua.
- **TTL pus separat de creare.** Daca aplicatia pica intre `INCR` si `EXPIRE`, cheia nu mai expira niciodata. Pune-le in aceeasi tranzactie sau acelasi script.
- **Nil si string-uri in Lua.** `GET` pe o cheie lipsa da `false`, iar pe una existenta da `"3"`. Ambele trebuie trecute prin `tonumber(x or '0')`.
- **Chei construite gresit.** Typo-uri de prefix (`r1:` in loc de `rl:`) sau obiecte interpolate in string. Pune prefixele intr-o singura clasa statica si verifica cu `SCAN` ce chei apar de fapt.

Plan de exersat, cam o ora pe zi:

1. **Ziua 1, String si chei.** In `redis-cli` exersezi `SET` / `GET` / `INCR` / `EXPIRE` / `TTL`. Construiesti de mana un fixed window si urmaresti cum expira.
2. **Ziua 2, Hash si Set.** Pui o regula in hash si un blocklist de IP-uri in set. Le citesti din C# cu `HashGetAll` si `SetContains`.
3. **Ziua 3, Sorted Set.** Faci de mana sliding window log-ul din sectiunea despre structuri, apoi il scrii ca script Lua.
4. **Ziua 4, atomicitate.** Rescrii fixed window cu `MULTI/EXEC`. Ii dai 50 de requesturi paralele dintr-un test xUnit si verifici ca exact `limit` trec.
5. **Ziua 5, token bucket in Lua.** Folosesti un Hash si `redis.call('TIME')`, cu `MONITOR` deschis cat rulezi.

Referinte: documentatia comenzilor la redis.io/docs/latest/commands si cea a clientului la stackexchange.github.io/StackExchange.Redis.